SRS

Blockchain-Based Secure Online Voting System

Muhammad Junaid

THE ISLAMIA UNIVERSITY OF BAHAWALPUR

(Information Technology)

Software Requirement Specifications (SRS)

Submitted by:

Muhammad Junaid (S23BINFT1M01013)

Submitted to:

Dr. Musarrat Kareem

Summary

This document presents the requirements for a **Blockchain-Based Secure Online Voting System.** The platform enables voters to cast votes securely from any device using blockchain technology. Every vote is stored immutably on the blockchain, ensuring transparency, tamper-resistance, and prevention of double-voting while supporting voter anonymity.

Unlike traditional paper-based or centralized voting systems that suffer from risks of rigging, tampering, and lack of trust, this system uses smart contracts to enforce one-person-one-vote rules and provides publicly verifiable results. The system is suitable for university elections, student councils, and organizational polls. It combines a modern web interface with decentralized backend logic for a trustworthy and transparent voting experience.

Table of Contents

1. Introduction

The proposed system is a decentralized web application (DApp) that allows secure online voting using blockchain technology. Traditional voting systems (paper ballots or centralized databases) are vulnerable to tampering, rigging, hacking, and lack of transparency. This system solves these issues by storing every vote immutably on the blockchain, enabling public verification of results without revealing voter identity.

The system provides a user-friendly interface for voters to connect their wallet, cast votes, and verify their votes, while administrators can manage candidates and control the election lifecycle.

1.1 Purpose

The purpose of this project is to design and develop a secure, transparent, and tamper-proof online voting system using blockchain. It aims to end double voting, prevent tampering, ensure voter anonymity, and provide real-time verifiable results, thereby increasing trust in the election process for university and organizational elections.

1.2 Scope

The system includes wallet-based authentication, voter registration, candidate management, secure vote casting, real-time result tallying, and vote verification. All critical voting logic and data are managed through smart contracts on the blockchain (Ethereum/Polygon testnet).

The system does not include biometric authentication, integration with national ID systems, or handling of large-scale national elections. It is focused on university levels and organizational levels.

1.3 Product Perspective

The system works as a decentralized application (DApp). It combines a responsive frontend interface for voting with smart contracts deployed on the blockchain. The frontend interacts with the blockchain through Web3/Ethers.js libraries. Votes are recorded directly on the blockchain, making the system transparent and resistant to manipulation.

1.4 User Characteristics

The system includes three main types of users:

- **Voters:** Students or organization members who connect their wallet to cast votes. They need basic knowledge of using MetaMask or similar wallets.
- **Admin:** Election organizer who manages candidates, starts/ends elections, and monitors results. Requires higher access privileges.
- **Public/Observer:** Anyone who can view public results and verify votes (read-only access).

1.5 Similar Apps and Systems / Literature Review

Existing online voting systems are mostly centralized and suffer from security and trust issues. Some blockchain-based voting prototypes exist in research but are often limited in features or usability. This project improves upon them by providing a complete DApp with wallet integration, smart contract enforcement of rules, real-time results, and vote verification — making it practical for real university elections.

1.6 Proposed Technologies

The system will be developed using the following technologies:

- **Blockchain:** Ethereum or Polygon, Solidity for smart contracts
- **Frontend:** React.js, HTML5, CSS3, JavaScript (ES6+)
- **Blockchain Interaction:** Ethers.js / Web3.js
- **Wallet Integration:** MetaMask
- **Backend**: Node.js + Express.js (for optional off-chain management)
- **Development Tools:** Hardhat, Ganache, VS Code, Git

2. Requirements

The system allows users to connect their wallet, register as voters, view candidates, cast votes securely, and view live results. Administrators can manage the election. All votes are processed and stored through smart contracts on the blockchain.

2.1 Functional Requirements

2.1.1 Connect Wallet

Name: FR001

Purpose: To authenticate users using cryptocurrency wallet

User(s): Voter, Admin

Input: MetaMask connection

Output: Wallet address retrieved and user authenticated

2.1.2 Voter Registration

Name: FR002

Purpose: To register voter linked with wallet address

User(s): Voter

Input: Wallet address

Output: Voter registered successfully

2.1.3 Manage Candidates (Admin

Name: FR003

Purpose: To add, update or remove candidates

User(s): Admin

Input: Candidate details (name, description, image)

Output: Candidates list updated on blockchain

2.1.4 Start/End Election (Admin)

Name: FR004

Purpose: To control election lifecycle

User(s): Admin

Input: Start and end timestamps

Output: Election status updated

2.1.5 Cast Vote

Name: FR005

Purpose: To allow voters to cast one vote

User(s): Voter

Input: Selected candidate

Output: Vote recorded on blockchain, voter marked as “Has Voted”

2.1.6 View Real-time Results

Name: FR006

Purpose: To display live vote counts

User(s): Voter, Admin, Public

Input: None (read-only)

Output: Live vote tally and candidate-wise results

2.1.7 Verify Vote

Name: FR007

Purpose: To let voters check if their vote was recorded

User(s): Voter

Input: Wallet address

Output: Verification confirmation

2.1.8 View Public Results

Name: FR008

Purpose: To provide transparent public access to results

User(s): Public

Input: None

Output: Election results displayed

2.2 Non-Functional Requirements

i. The system should respond to user actions with minimal delay (transaction time acceptable on testnet).
ii. The interface should be fully responsive across desktop, tablet, and mobile devices.
iii. All votes must be stored immutably on the blockchain with strong security against tampering and double voting.
iv. The system should support voter anonymity while allowing public verification of results.
v. The DApp should manage university-level elections (up to one thousand voters) efficiently.
vi. Smart contracts must be gas-efficient and follow security best practices.

2.2.1 Security Requirements

All votes must be stored immutably on the blockchain.

The system must prevent double-voting and unauthorized vote modification.

Voter anonymity must be preserved (wallet address should not directly reveal real identity).

Protection against common smart contract vulnerabilities (reentrancy, access control, overflow).

2.2.2 Performance Requirements

Page load time should be ≤ 3 seconds.

Transaction confirmation time should be reasonable for testnet (preferably under 30 seconds on Polygon).

2.2.3 Usability Requirements

The user interface should be clean, modern, and responsive.

Clear instructions and tooltips shall be provided for wallet connection and voting process.

Color scheme: Professional blue and green tones.

2.2.4 Reliability & Availability

The system shall remain available 24/7 during the election period (subject to blockchain network status).

Election results must remain accessible after the election ends.

2.2.5 Scalability

The system should efficiently support at least five hundred–one thousand voters (suitable for university-level elections).

2.2.6 Maintainability

Smart contracts shall be modular, well-commented, and follow Solidity best practices.

Source code shall be properly version-controlled using Git.

3. Use Cases and Flow of Processes

The system works through interactions between users and the decentralized application. Each case describes how a user performs a task and how the system responds.

3.1 Use Case 1

Field	Description
ID	UC001
Name	Cast Vote
DESCRIPTION	This case describes casting of vote
Requirements	FR005
Actor	Voter
Precondition	In this section wallet connection and voter registration has discussed

How voting works for the user:

1. You connect your wallet and sign in

Open the app, click “Connect Wallet,” and approve the login with MetaMask. That is how we know it is really you.

2. You see who is running

Once you are in, the screen shows all the candidates with their names and photos. No scrolling through code — just a simple list.

3. You pick your candidate

Tap the one you want. You can change your mind until you hit confirm**.**

4. We make sure you have not voted already

Behind the scenes, the system checks: one person, one vote. If you have already voted, it will tell you and stop you here.

5. You confirm in MetaMask

A MetaMask popup shows up asking you to approve the vote. This is what makes it official and locks it on the blockchain.

6. Your vote gets recorded

The smart contract takes your choice and stores it permanently. No one can change it or see who you picked.

7. You get a "You're done!" message

The screen shows success confirmation, so you know your vote counted**.**

Exceptions

- Election has not started or already ended.
- Voter has already voted.
- Transaction rejected in wallet
- Insufficient gas or network error

3.2 Use Case Diagrams:

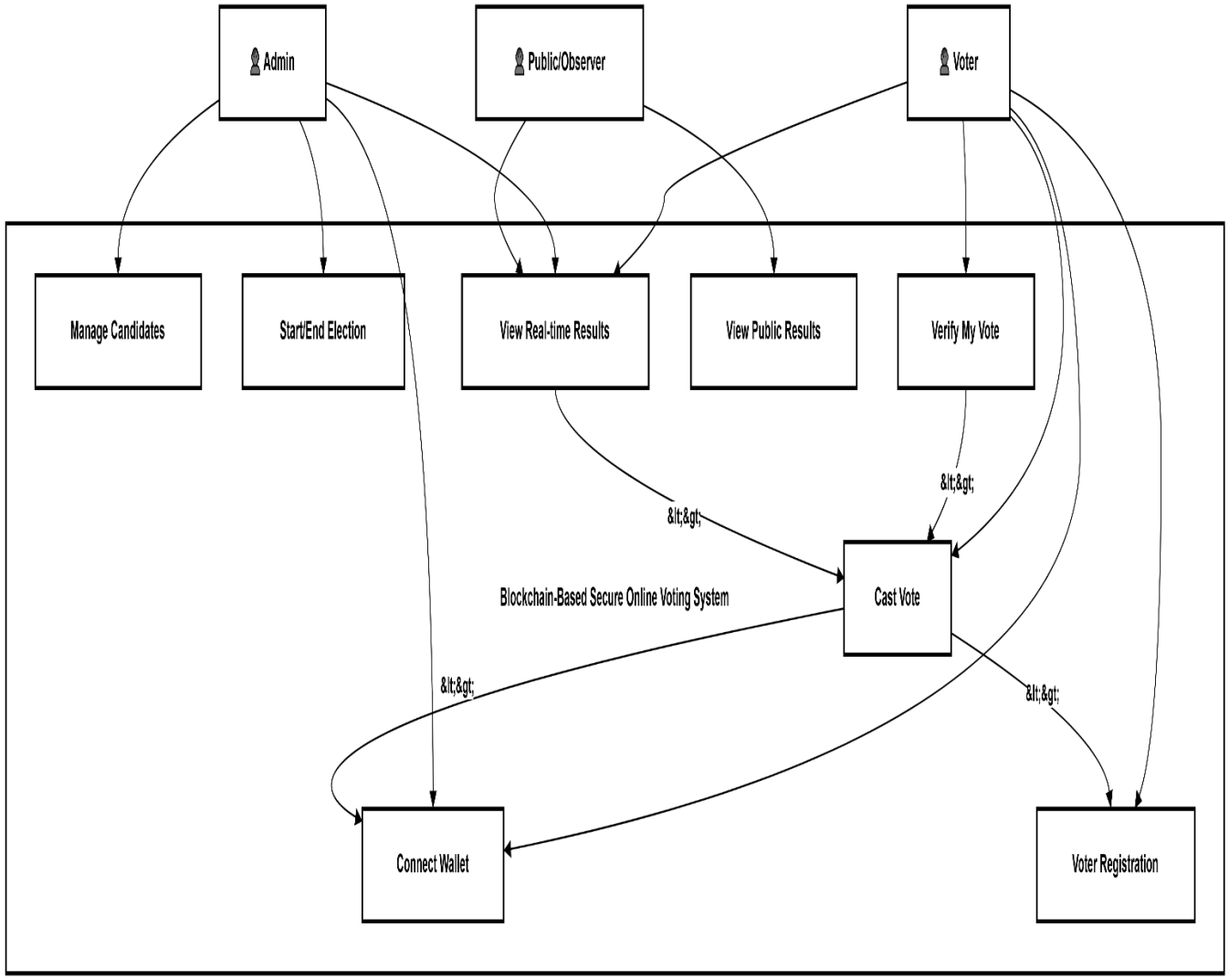


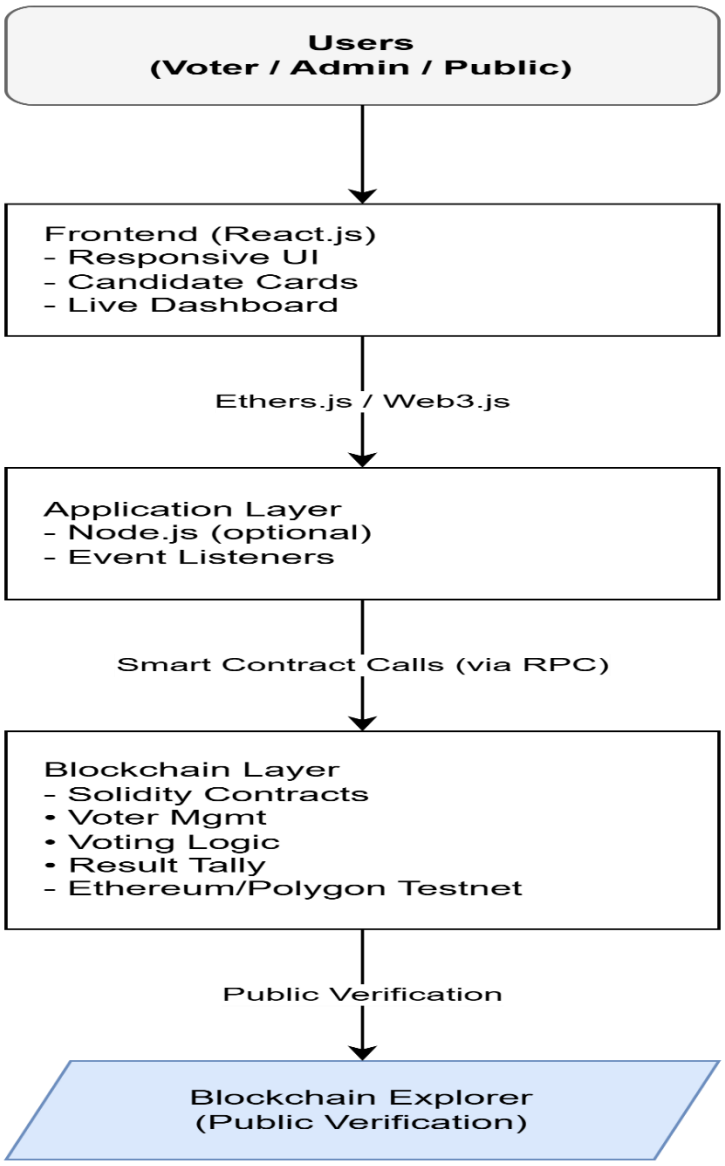
Fig: 01: Voting System

Description:

The diagram shows the main actors (Voter, Admin, and Public/Observer) and their interactions with the system. "Cast Vote" is the central use case that includes "Connect Wallet" and "Voter Registration". "Verify My Vote" and "View Real-time Results" extend from the voting process. Admin has special privileges for managing candidates and controlling the election.

.

3.3 System Architecture Diagram:



4. References

- IEEE Guide for Software Requirements Specifications
- Solidity Documentation and Ethereum Developer Resources
- Web3.js / Ethers.js Official Documentation
- Research papers on Blockchain-based Voting Systems
- Hardhat and Polygon Developer Documentation